Secrets Management

never in code, never forever

Watch the 4-min video

In one line: keep secrets out of code, images and config; let each workload fetch them at runtime from a secret store using its own identity, prefer short-lived dynamic credentials, rotate the rest automatically, and scan every commit.

1 Leaked in minutes

A developer pushes a quick test to a public repo with a database password inside. Bots scan public GitHub continuously, and the credential is being tried within minutes. Secrets in code, config files and container images are one of the most common ways in: cheap to find, expensive to clean up. Deleting the line doesn't help; **Git history keeps it forever**, and every clone and fork has a copy.

2 Where secrets should (not) live

Location	Problem
Source code, Git history	copied to every clone, fork and CI cache; public repos are scanned by bots
Container image, plain config file	pushed to registries, readable by anyone who can pull the image
Environment variables	better, but leak via crash dumps, debug pages, child processes and `/proc`
Kubernetes Secret	only base64 in etcd unless encryption at rest is enabled; anyone with `get secrets` reads them
Secret store (Vault, AWS Secrets Manager, Azure Key Vault, GCP Secret Manager)	encrypted, per-secret access policy, audit log, versions, rotation

The application authenticates to the store with its **workload identity** (lesson 45), not with yet another secret, and fetches at startup or on demand. Platforms can also mount secrets as files (Secrets Store CSI driver, Vault Agent) or sync them (External Secrets Operator).

3 Dynamic secrets

Instead of one long-lived password shared by every instance, the store **creates credentials on request**. A leaked credential expires on its own, and every database session traces back to one instance. Cloud equivalents: IAM database authentication on RDS, or Entra ID tokens for Azure SQL, which remove the password entirely.

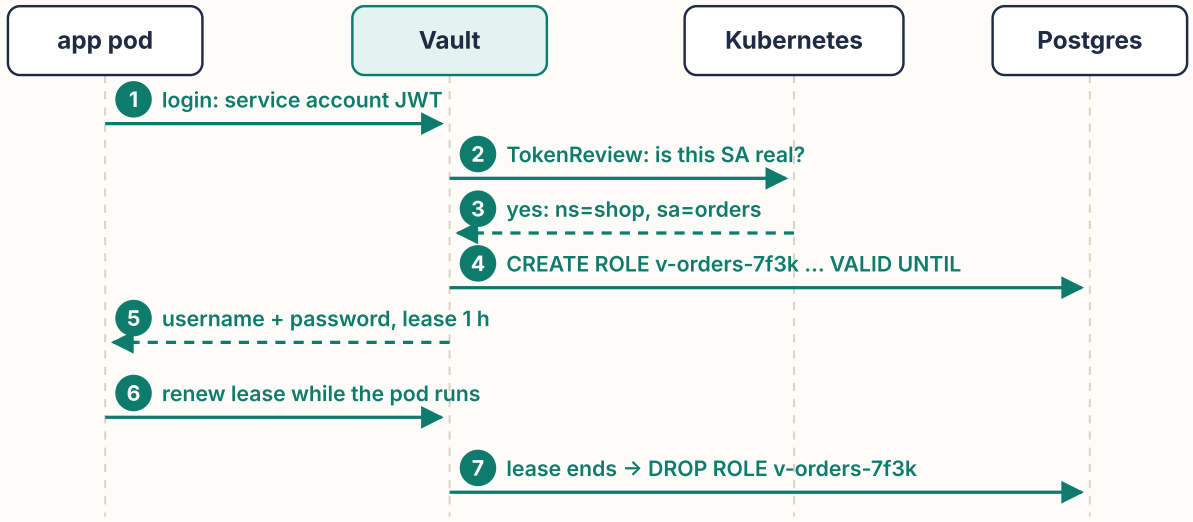


Figure 1. Vault's database secrets engine creates a unique database user per app instance with a one-hour lease, then deletes it.

4 Rotation without outages

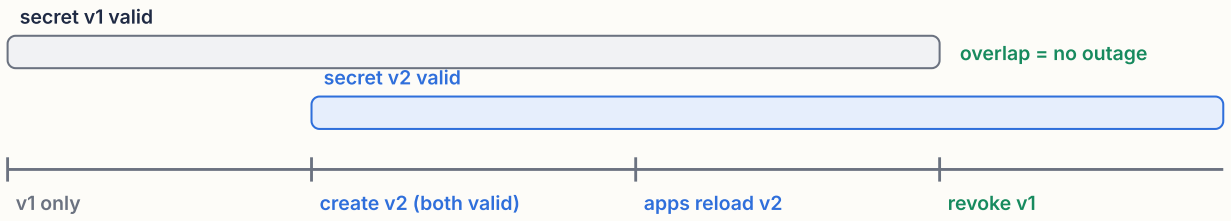


Figure 2. Rotate with two valid versions: create the new one, let every instance pick it up, then revoke the old one.

Static secrets (third-party API keys, legacy DB users) still exist, so rotate them on a schedule and automatically: AWS Secrets Manager runs a rotation function; the “alternating users” strategy keeps two DB users and flips between them. Apps should reload secrets without a restart, or the platform should roll pods gracefully. Track each secret's age and alert on old ones.

5 Catch mistakes early

Assume someone will slip. **Pre-commit hooks** (gitleaks, detect-secrets) stop secrets on the laptop; **GitHub secret scanning** with push protection blocks known key formats on push and alerts on history; CI scans images too. If a secret leaks: **revoke and rotate immediately**, then check its audit trail for use. Rewriting history is cleanup, not remediation.

WORKED EXAMPLE: VAULT ROTATING DATABASE CREDENTIALS

The pod authenticates to Vault with its Kubernetes service account (Kubernetes auth method). A Vault role maps `shop/orders` to a database role with `default_ttl=1h`. Vault creates `v-orders-7f3k` in Postgres and returns its credentials. The Vault Agent sidecar writes them to a shared in-memory file, renews the lease while the pod runs, and fetches new ones before the maximum TTL. Nobody sees the password, nothing is in Git, and deleted pods leave no valid credentials behind.

COMMON MISTAKES

- Deleting the leaked line in a new commit and calling it fixed.
- Bootstrapping the secret store with a static token baked into the image (“secret zero”).
- One shared credential for all services and environments, so nothing can be traced or rotated safely.
- Logging configuration or request headers, which prints the secrets you carefully protected.

6 Key terms

Secret: anything that grants access: password, API key, private key, token

Secret store: encrypted, audited service for secrets

Dynamic secret: credential created on demand with a short lease

Lease / TTL: how long a credential stays valid

Secret zero: the first credential needed to reach the store

Secret scanning: detecting keys in commits, history and images

7 Interview questions

Q1. How should an app get its database password? BASIC

Not from code, images or plain config: it authenticates to a secret store with its workload identity and fetches the credential at runtime, ideally a short-lived dynamic secret that is rotated automatically.

Q2. What else helps besides a secret store? BASIC

Least-privilege access policies per secret, audit logs, and secret scanning on every commit; if a secret leaks, revoke and rotate it immediately.

Q3. An AWS key was pushed to a public repo an hour ago and then deleted. What do you do? SCENARIO

Treat it as compromised: deactivate it now, issue a replacement, review CloudTrail for its use, then purge history and add push protection.

Q4. How do you rotate a database password used by 40 running pods with no downtime? SCENARIO

Create a second valid credential, roll or hot-reload the pods onto it, verify, then revoke the old one; or move to dynamic per-pod credentials.

Q5. How does the app authenticate to the secret store without a secret? TRICKY

Using an identity the platform vouches for: a Kubernetes service account token, cloud managed identity or instance role, verified by the store.

Q6. Are environment variables a safe place for secrets? TRICKY

Better than code, but they leak through crash reports, debug endpoints, child processes and logs. Mounted files or direct SDK fetches with short lifetimes are safer.

Next: SD-M-011 · Hashing Passwords backendroadmap.com